
LemmaPortfolio: Choosing One Small Lemma Set for Many Lean Goals

Anonymous Author(s)

Affiliation

Address

email

Abstract

Premise selection normally asks which lemmas may help one proof state. We study the shared-budget problem: which small lemma set best serves several related goals at once? A LEMMAPORTFOLIO episode displays the types of 16 name-masked candidates and eight name-masked targets, and asks for three candidates. A target is covered when at least one chosen candidate occurs directly in its historical elaborated value. We construct 30 development and 60 test episodes from MATHLIB v4.33.0, enumerate all 560 portfolios, retain every maximizer, and independently replay the dependency labels in pinned Lean. The public test set and label digest were committed before the response document was supplied. Among five supplied consumer-response rows, exact-optimum accuracy ranged from 1/60 to 25/60; the best row covered 299/480 targets but attained an optimum on only 41.7% of episodes. On 20 matched items, a support-prediction decomposition attained 1/20 exact optima, compared with 4/20 under direct prompting, locating a concrete failure mode rather than an improvement. The task measures compression of historical dependencies, not proof success under added premises.

1 Introduction: one budget, several goals

Suppose that a prover can place only three lemmas in a shared context for eight related goals. Choosing a plausible lemma for each goal independently can waste that budget: a less obvious lemma may be useful across several goals. The same choice arises when building a retrieval cache for a file or asking a mathematician to inspect a short list of reusable lemmas. We call this shared selection problem the *lemma-portfolio* problem.

Most premise retrievers are queried for one proof state. LeanDojo and ReProver make this setting explicit for Lean [Yang et al., 2023], Magnushammer studies the analogous problem for Isabelle [Mikuła et al., 2024], and recent Lean work also uses the dependency graph [Petrovčič et al., 2025]; LeanHammer adapts neural selection to local contexts and couples it to proof reconstruction [Zhu et al., 2026]. UseFor asks the stronger utility question of how a theorem changes proof likelihood [Kasriel et al., 2025]. LeanSearch v2 retrieves a premise group for one whole theorem [Gao et al., 2026], while miniCTX studies theorem proving with long project context [Hu et al., 2025]. TheoremBench expands one theorem into related proving tasks and reports partial progress [Pham et al., 2026], but does not impose one shared premise budget across them. We instead isolate the finite decision that comes before several proofs: one fixed-size portfolio from a closed pool must serve all of them. To our knowledge, the formal-premise benchmarks reviewed above do not score this multi-target, shared-budget objective with complete exact optima.

The system must infer a hidden pattern of historical reuse from formal statements alone. Because the pool is small, we can enumerate every portfolio and credit every maximizer rather than privilege one answer. Our contribution is threefold: a precise shared-budget objective, a module-disjoint and

37 exactly audited MATHLIB benchmark, and an evaluation that separates exact portfolio choice from
 38 partial coverage and support prediction. The benchmark is not a new prover or a claim about logical
 39 necessity.

40 2 Task, scoring, and construction

41 An episode contains a candidate pool C with $|C| = 16$, a target set T with $|T| = 8$, and a budget
 42 $B = 3$. Candidate and target declaration names are replaced by identifiers C01–C16 and T01–T08;
 43 candidate types and target statements remain visible. We use “lemma” informally for a candidate
 44 declaration, which may also be a definition or constructor. For a target t , let

$$D(t) = \{c \in C : c \text{ occurs as a direct constant in the elaborated value of } t\}.$$

45 Occurrence is syntactic in the elaborated declaration body; we take no transitive closure. For a
 46 theorem, this body is its elaborated proof term. For a portfolio $S \subseteq C$ of size B , its historical target
 47 coverage is

$$F(S) = |\{t \in T : D(t) \cap S \neq \emptyset\}|. \quad (1)$$

48 Let $F^* = \max_{|S|=B} F(S)$ and $\mathcal{A} = \{S : |S| = B, F(S) = F^*\}$. We enumerate all $\binom{16}{3} = 560$
 49 portfolios and reconstruct $\mathcal{A}$. The primary test score is the number n of the 60 episodes for which the
 50 submitted portfolio is in $\mathcal{A}$, reported as $n/60$ and a percentage. Thus every maximizing portfolio
 51 is correct. The secondary item scores are raw target coverage $F(S)$ (also $F(S)/8$), normalized
 52 coverage $F(S)/F^*$, and direct-edge recall

$$R_E(S) = \frac{\sum_{t \in T} |D(t) \cap S|}{\sum_{t \in T} |D(t)|}.$$

53 Aggregate edge recall is micro-averaged by summing this numerator and denominator over episodes.
 54 A missing, malformed, wrong-budget, duplicate-ID, or out-of-pool response gets zero on all four
 55 scores and remains in the 60-item denominator; validity is reported separately.

56 This is a small maximum-coverage problem [Nemhauser et al., 1978], but the incidence graph is
 57 hidden behind the displayed statements. If C01 serves three targets, C02 two others, and C03 a sixth,
 58 their portfolio covers six; spending two slots on alternatives for the first three targets can waste the
 59 budget. The 16/8/3 design keeps prompts readable and all choices enumerable while still rewarding
 60 reuse.

61 **Source and construction.** We mine target values and candidate types from MATHLIB v4.33.0 at
 62 commit db584cd6d46c92f209a44c0f1c829460d327499d [The mathlib Community, 2020, 2026].
 63 Each episode comes from one module, with names hidden. Development, test, and all earlier prototype
 64 modules and statements are disjoint. We reject simple “largest star” instances: each retained episode
 65 has at least seven active candidates, at least four active candidates outside every optimum, multiple
 66 multi-edge targets, and multiple near-optimal portfolios; every member of an optimum is reused and
 67 essential to that portfolio. The complete thresholds are in the supplement.

68 The miner also rejects any episode solved by any of 20 fixed construction methods. Eighteen use
 69 only public strings or positions; two use the true incidence graph and rank candidates by frequency or
 70 greedy maximum coverage. These methods define the challenge distribution and are not held-out
 71 baselines. Among the 60 test episodes, 45 have $F^* = 6$ and 15 have $F^* = 7$; together they have 78
 72 accepted optimal portfolios. The final split contains 30 development and 60 test episodes across 15
 73 top-level domains.

74 **Commitment and label audit.** The underlying theorem statements are public and may occur in
 75 model training, so we do not claim contamination-free evaluation. What is new is their composition
 76 into name-masked episodes and the hidden incidence rows. On 1 September 2026 we recorded
 77 the public-file hash and a digest of the label file; the response PDF used below was supplied on 2
 78 September. The prompts do not contain the labels. A separate pinned-Lean 4.33.0 replay reconstructed
 79 every displayed edge, and scoring freshly reconstructs every optimum. The response PDF lacks
 80 provider logs and per-chat timestamps, so the deployment rows are external captures with a prior
 81 dataset commitment, not a cryptographic blind evaluation. The supplement states this evidence
 82 boundary precisely.

Table 1: V4 test results. The primary score credits an episode only when the selected triple attains the exact maximum coverage; mean target coverage gives partial credit. Both use the fixed 60-episode denominator.

System or deployment label	Exact optimum	Mean coverage	
	$n/60$ (%)	$F(S)/8$ (%)	Valid
Uniform random (analytic expectation)	0.14/60 (0.2)	2.35/8 (29.3)	–
Character TF-IDF	0/60 (0.0)	3.77/8 (47.1)	60/60
GPT SOL 5.6 Pro	25/60 (41.7)	4.98/8 (62.3)	55/60
DeepSeek Instant + DeepThink	1/60 (1.7)	4.20/8 (52.5)	60/60
DeepSeek Expert + DeepThink	7/60 (11.7)	4.60/8 (57.5)	60/60
Qwen 3.8 Max-Thinking	12/60 (20.0)	4.32/8 (54.0)	55/60
Qwen 3.7 Plus-Thinking	6/60 (10.0)	4.05/8 (50.6)	60/60

3 Experimental setup

We ask three questions. How often does a displayed deployment select an exact optimum? How much of the target set does it cover when it misses? Finally, does making the hidden target–candidate relation explicit improve the joint decision? The author-supplied document reports five visible consumer labels: GPT SOL 5.6 Pro, two DeepSeek DeepThink modes, and Qwen 3.8 Max and 3.7 Plus in Thinking mode. Each direct row is intended to cover all 60 episodes in twelve five-item prompts. We align responses by their explicit episode IDs, not by page position, preserve malformed or missing blocks as zeroes, and make no repair. The analysis retains at most one supplied capture per intended block; anomalies remain zero-scored, and no repeated sampling is available.

The sole model-facing development check used the first 15 development episodes: a controlled Codex CLI Sol capture obtained 6/15 exact optima. It motivated no change to the committed test set and is not pooled with the consumer rows.

Consumer interfaces do not expose the serving checkpoint, system prompt, decoding policy, or routing decisions. We therefore report the visible labels verbatim. The convenience sample is neither representative nor a ranking of providers or model families.

Two non-model references were fixed in advance. Under uniform selection, every portfolio has probability $1/560$, so the probability of an optimum is $|\mathcal{A}|/560$. If $d_t = |D(t)|$, the expected episode coverage is

$$\mathbb{E}[F(S)] = \sum_{t \in T} \left(1 - \frac{\binom{16-d_t}{3}}{\binom{16}{3}} \right),$$

and expected edge recall is $3/16$; no Monte Carlo estimate is needed. A character 3–5-gram TF-IDF reference instead enumerates all portfolios and maximizes summed target-to-candidate facility-location similarity. It was run only after the split commitment and never used to admit an item. The supplement gives the full calculations.

4 Results

The exact and partial-credit columns tell different parts of the story. GPT SOL had the largest exact score, 25/60, and covered 299 of the 480 displayed targets; its mean oracle-normalized coverage was 79.6%. Qwen 3.8 Max had the next largest exact score, 12/60. The remaining exact scores were 7/60 for DeepSeek Expert, 6/60 for Qwen 3.7 Plus, and 1/60 for DeepSeek Instant. Thus a system could often identify useful candidates without assembling a maximizing three-candidate portfolio. Character TF-IDF likewise covered 47.1% of targets but found no exact optimum, which indicates that the task is not solved by this surface-similarity rule.

Two five-item response blocks were invalid under strict whole-block parsing: the GPT document repeats the block for episodes 0050–0054 and supplies no direct block for 0055–0059, while the Qwen 3.8 block for 0055–0059 lacks its opening outer brace. We neither repair nor remove these outcomes. Episode keys, rather than the visual block order, align every other response to its prompt. Wilson and whole-block bootstrap intervals, per-block counts, and the full transcription audit appear in the supplement. These convenience-sample rows identify dated consumer labels, not reproducible checkpoints or model families.

120 5 Diagnosing errors with structured support prediction

121 A direct portfolio does not show whether Sol inferred the target–candidate relations incorrectly or
122 combined plausible relations poorly. On 20 test episodes, four additional responses return up to four
123 candidate IDs for each target, ordered by plausibility. A deterministic program keeps the first two per
124 target ($q = 2$), then exactly solves the induced size-three coverage problem; ranks three and four are
125 used only to diagnose the inferred incidence graph.

126 On the same 20 public episode IDs, direct GPT SOL prompting attained 4/20 exact optima, whereas
127 ordered support prediction followed by the fixed $q = 2$ optimizer attained 1/20, a descriptive change
128 of -15 percentage points. The retained support edges nevertheless had 76.8% micro precision and
129 68.7% micro recall. This particular decomposition therefore produced a reasonably informative
130 incidence sketch but a worse exact portfolio; it does not support an improvement claim.

131 This is an error-localization diagnostic, not a second model. It compares direct prompting with
132 the complete structured-prompt-plus-solver pipeline. We report item and whole-block differences
133 without a significance test. Since the source document does not certify paired-call chronology, the
134 contrast is descriptive rather than causal.

135 6 Limitations and ways forward

136 LEMMAPORTFOLIO compresses one historical direct-dependency graph. Direct occurrence is
137 reproducible, but it is neither logical necessity nor evidence that supplying a lemma will cause a new
138 proof to succeed. The pool is closed, all costs are uniform, and the 60 modules form a deliberately
139 conditioned stress set. Public statements may have appeared in training. Consumer routing is opaque,
140 the response document supplies no per-call operational evidence, and we have one pass and no
141 human-expert baseline. Intervals therefore describe this fixed set; they do not estimate performance
142 over MATHLIB or generation variance.

143 The natural next step is to train and evaluate set-level retrievers rather than assemble per-goal rankings
144 after the fact. Predicted incidence can still be useful, but the $q = 2$ result shows that calibration,
145 truncation, and tie-breaking must be studied jointly. Larger candidate pools, nonuniform context
146 costs, causal proof-success labels, repeated runs, and a mathematician baseline would move the
147 benchmark closer to prover practice.

148 7 Conclusion

149 Choosing one shared budget across several goals is a finite decision distinct from per-goal retrieval,
150 and it admits exact scoring with every tie preserved. The observed gap between partial coverage and
151 exact optimality shows that recognizing individually plausible premises does not by itself solve the
152 shared-budget decision. The supplement and anonymous artifact provide the data, released labels,
153 response transcription, scorer, Lean replay receipts, and reproduction commands.

References

- Guoxiong Gao, Zeming Sun, Jiedong Jiang, Yutong Wang, Jingda Xu, Peihao Wu, Bryan Dai, and Bin Dong. LeanSearch v2: Global premise retrieval for Lean 4 theorem proving, 2026. URL <https://arxiv.org/abs/2605.13137>.
- Jiewen Hu, Thomas Zhu, and Sean Welleck. miniCTX: Neural theorem proving with (long-)contexts. In *International Conference on Learning Representations*, 2025. URL <https://openreview.net/forum?id=KIgaAqEFHW>.
- Timothe Kasriel, Thomas Lu, Devon Ding, Jingxuan He, and Dawn Song. Usefulness-driven learning of formal mathematics. In *NeurIPS Workshop on Mathematical Reasoning and AI*, 2025. URL <https://openreview.net/pdf/023eb56eadd5685c15a6489e0fddd540279b8c45.pdf>.
- Mathlib Initiative. Mathlib constant dependencies. Hugging Face dataset, 2026a. URL <https://huggingface.co/datasets/mathlib-initiative/mathlib-const-dep>. Version 4.33.0; exact 256-shard inventory recorded in the artifact.
- Mathlib Initiative. Mathlib types. Hugging Face dataset, 2026b. URL <https://huggingface.co/datasets/mathlib-initiative/mathlib-types>. Version 4.33.0; exact 256-shard inventory recorded in the artifact.
- Maciej Mięk, Szymon Tworowski, Szymon Antoniak, Bartosz Piotrowski, Qiaochu Jiang, Jin Zhou, Christian Szegedy, Łukasz Kuciński, Piotr Miłoś, and Yuhuai Wu. Magnushammer: A transformer-based approach to premise selection. In *International Conference on Learning Representations*, 2024. URL https://proceedings.iclr.cc/paper_files/paper/2024/hash/ab5f5f22e3e09f4424592fffb06840ab0-Abstract-Conference.html.
- George L. Nemhauser, Laurence A. Wolsey, and Marshall L. Fisher. An analysis of approximations for maximizing submodular set functions—i. *Mathematical Programming*, 14:265–294, 1978. doi: 10.1007/BF01588971.
- Job Petrovič, David Eliecer Narvaez Denis, and Ljupčo Todorovski. Combining textual and structural information for premise selection in Lean. In *NeurIPS Workshop on Mathematical Reasoning and AI*, 2025. URL <https://arxiv.org/abs/2510.23637>.
- QuocViet Pham, Elvir Karimov, Andrey Galichin, and Ivan Oseledets. TheoremBench: Evaluating LLMs on theorem proving in formal mathematics, 2026. URL <https://arxiv.org/abs/2606.09450>.
- The mathlib Community. The Lean mathematical library. In *Proceedings of the 9th ACM SIGPLAN International Conference on Certified Programs and Proofs*. ACM, 2020. doi: 10.1145/3372885.3373824.
- The mathlib Community. mathlib4 source snapshot for version 4.33.0. Git repository commit, 2026. URL <https://github.com/leanprover-community/mathlib4>. Commit db584cd6d46c92f209a44c0f1c829460d327499d.
- Kaiyu Yang, Aidan M. Swope, Alex Gu, Rahul Chalamala, Peiyang Song, Shixing Yu, Saad Godil, Ryan Prenger, and Anima Anandkumar. LeanDojo: Theorem proving with retrieval-augmented language models. In *Advances in Neural Information Processing Systems*, volume 36, 2023. doi: 10.52202/075280-0944. URL https://proceedings.neurips.cc/paper_files/paper/2023/hash/4441469427094f8873d0fecb0c4e1cee-Abstract-Datasets_and_Benchmarks.html.
- Thomas Zhu, Joshua Clune, Jeremy Avigad, Albert Qiaochu Jiang, and Sean Welleck. Premise selection for a Lean hammer. In *International Conference on Learning Representations*, 2026. doi: 10.48550/arXiv.2506.07477. URL <https://openreview.net/pdf/0d32353ccfa85cbf700389ec2d98a2c40fa4b640.pdf>.

Technical supplement

This supplement specifies the benchmark, construction, response transcription, and artifact trail behind the four-page paper. It is self-contained for score reproduction: every score-bearing claim can be traced from the supplied response PDF and episode-key transcription to a public episode, a committed label row, and deterministic scoring code. The accompanying anonymous archive contains those files and the verification commands. It also records what the evidence does *not* establish.

A Task definition

For each episode, let C be the displayed candidate set, T the displayed target set, and B the selection budget. The released benchmark fixes $|C| = 16$, $|T| = 8$, and $B = 3$. Candidate and target identifiers are local to an episode; a name such as C04 has no meaning in another episode. Candidate and target declaration names are replaced by these identifiers, but the canonical candidate types and target statements remain visible.

For each target t , the label records

$$D(t) = C \cap \text{Consts}(\text{value}(t)),$$

where `value` is the elaborated Lean declaration value (including opaque theorem values in the audit) and `Consts` returns the constants occurring directly in its expression tree. The intersection is important: a target may use other declarations that are not among the 16 displayed candidates. No transitive closure is taken.

For an exact-budget portfolio $S \subseteq C$, define

$$F(S) = \sum_{t \in T} \mathbf{1}[D(t) \cap S \neq \emptyset].$$

The scorer enumerates every $S \subseteq C$ with $|S| = B$ in lexicographic order, computes $F^* = \max_S F(S)$, and reconstructs the complete set $\mathcal{A} = \{S : F(S) = F^*\}$. A response is correct precisely when it is a valid size- B subset in $\mathcal{A}$. The stored optimum and stored argmax list are treated as assertions to audit, not as authority: scoring stops if either differs from the fresh enumeration.

The three secondary item scores are raw target coverage $F(S)$ (reported in aggregate also as $F(S)/|T|$),

$$\text{normalized coverage} = F(S)/F^*, \quad \text{edge recall} = \frac{\sum_t |D(t) \cap S|}{\sum_t |D(t)|}.$$

A missing, malformed, wrong-budget, duplicate-ID, or out-of-pool response gets zero on the primary exact-optimum score and all three secondary scores. Validity and operational status are reported separately.

A.1 Exact uniform-random reference

There are $\binom{K}{B}$ equally likely portfolios for $K = |C|$. Hence, for one episode,

$$\Pr(S \text{ is optimal}) = \frac{|\mathcal{A}|}{\binom{K}{B}}.$$

For a target with $d_t = |D(t)|$, a random portfolio misses all its displayed dependencies with probability $\binom{K-d_t}{B}/\binom{K}{B}$ (taken as zero when $K - d_t < B$). Linearity of expectation therefore gives

$$\mathbb{E}[F(S)] = \sum_{t \in T} \left(1 - \frac{\binom{K-d_t}{B}}{\binom{K}{B}} \right).$$

We average the item-level optimal probabilities and $\mathbb{E}[F(S)]/F^*$. Since each displayed edge is incident to one candidate, its selection probability and the expected edge recall are both $B/K = 3/16$. No Monte Carlo estimate is needed. A single seeded SHA-256 draw is retained only as a parser and provenance check.

B Source snapshot and episode construction

B.1 Pinned source

The construction begins from MATHLIB v4.33.0 [The mathlib Community, 2026]. Its pinned Git commit is

db584cd6d46c92f209a44c0f1c829460d327499d.

The extraction consists of 256 Parquet shards from the Mathlib Initiative dependency and type datasets [Mathlib Initiative, 2026a,b]. Before mining, the builder recomputes the name, byte size, and SHA-256 digest of every shard and compares the complete inventory with the earlier source receipt. It refuses a missing, extra, reordered, resized, or changed shard. The generation receipt records the complete inventory rather than relying on a directory-level label.

Score reproduction uses the released episode and label JSONL files and does not require these Parquet shards. The pinned-Lean audit can recompute every displayed edge from the named MATHLIB commit. Repeating the original mining search, however, additionally requires the separately archived 256-shard snapshot; its per-file inventory detects alteration but cannot recover missing bytes.

Eligible declarations have a nonempty, parseable public type, a source module, and a proof-body dependency record. We remove source modules used by any earlier LemmaPortfolio prototype and reject any statement whose whitespace-normalized or leading-binder- α -normalized signature appeared in those prototypes. The same two signatures are used to forbid duplicates within and between the new development and blind splits.

B.2 Cohorts, targets, and masks

Each episode is centered on one source module. Candidate cohorts are drawn from declarations that recur as dependencies elsewhere in the corpus. Targets come from the same module and have at least one edge to the cohort. Stable SHA-256 ranks, fixed in the builder, determine cohort trials, target trials, candidate display order, target display order, and final split order. Human or model responses are not inputs to any of these ranks.

The public row contains only the benchmark ID, local candidate and target IDs, canonical statement strings, budget, task text, and source-project label. The released label row maps local IDs back to source declarations and records every $D(t)$, candidate use counts, F^* , and every optimal portfolio. Before accepting a row, the builder rejects any source theorem name visible in any displayed statement. It also rejects exact or α -equivalent candidate–candidate, candidate–target, and target–target collisions.

B.3 Structural stress filters

Table 2 lists the fixed filters applied before final split selection. They were chosen on development experience and then applied identically to development and blind candidates. A candidate is active if it belongs to at least one $D(t)$; “near-optimal” means coverage $F^* - 1$.

Table 2: Episode-level construction requirements.

Requirement	Threshold
Displayed candidates / targets / budget	16/8/3
Active candidates	at least 7
Active candidates outside the union of all optima	at least 4
Targets with two or more displayed direct dependencies	at least 2
Coverage of any singleton candidate	at most 4 of 8
Use count of every member of an optimal portfolio	at least 2
Loss after removing any member of an optimal portfolio	at least 1 target
Distinct portfolios at coverage $F^* - 1$	at least 2
Optimal coverage	at least 6 of 8
Number of accepted optimal portfolios	at most 32
Mean pairwise target token-Jaccard similarity	at most 0.6

The miner further requires 20 fixed methods to miss the optimum. Eighteen use only public strings: candidate position or length ranks, token-count and connective ranks, aggregate candidate–target text similarity, and three token-Jaccard inferred-incidence widths. Two use the true incidence graph: frequency rank and greedy maximum coverage. This conditioning is disclosed in the public generation receipt. These methods describe how the challenge set was selected and must not be presented as held-out baselines.

The post-freeze character TF–IDF baseline is different in both timing and implementation. It is specified before the final commitment but generated afterward, reads public statements only, and never enters item admission. It uses character n -grams for $n \in \{3, 4, 5\}$, sublinear term frequency $1 + \ln c$, smoothed inverse document frequency $\ln((1 + N)/(1 + \text{df})) + 1$, fitted within each episode over its 24 candidate and target statements, L2 normalization, and cosine similarity. It enumerates all size-three portfolios to maximize $\sum_t \max_{c \in S} \text{sim}(t, c)$, breaking ties lexicographically. Because both methods use surface text, we still describe it as related to the lexical diagnostic family.

Table 3: Final split summary after the pinned-Lean replacement.

	Development	Blind
Episodes / distinct modules	30 / 30	60 / 60
Top-level domains	13	15
Displayed targets	240	480
Displayed direct edges	386	814
Multi-edge targets	99	226
Mean active candidates	8.37	8.60
$F^* = 6 / F^* = 7$	25 / 5	45 / 15
Accepted optimal portfolios	30	78
Construction-diagnostic hits	0	0

The miner began with 350,599 declarations. After eligibility and prior-split exclusions, 186,680 declarations remained across 7,089 module centers. Stable cohort and target trials produced 71,091 structurally complete incidence drafts. Of these, 165 made all 20 construction diagnostics miss, 121 also passed name masking, and 103 also passed within-episode statement identifiability. One initially selected blind module failed the independent Lean edge audit and was excluded by its module hash; the final row was chosen deterministically from the already ranked admissible superset.

The final blind domain counts are: Algebra 10, Data 7, Ring Theory 7, Analysis 5, Topology 5, Algebraic Geometry 4, Category Theory 4, Combinatorics 3, Number Theory 3, Order 3, Dynamics 2, Geometry 2, Group Theory 2, Linear Algebra 2, and Measure Theory 1. The largest domain therefore contributes 10/60 episodes.

C Development policy and dataset commitment

Thirty development episodes are partitioned in manifest order into two predeclared, nonoverlapping batches of 15. Batch A supplies the sole model-facing difficulty check: a controlled Codex CLI Sol capture obtained 6/15 exact optima (40%). This single check suggested that the revised format was nontrivial. It is development evidence only, not a consumer-chat test result, an upper bound, or a guarantee that blind Sol performance will be below 60%. Batch B is not used for additional model calibration, and neither batch is used to choose the ordered-support width. No development item is substituted for a blind item. All development labels are public and excluded from blind prompt directories.

The 60 test episodes are selected by the same structural rules from disjoint modules, with a fixed top-level-domain cap. The source theorems remain public; “hidden” refers to the newly composed episode and its incidence row. On 1 September 2026, before the response document was supplied to the evaluator, the project recorded the public test SHA–256 and label-file commitment shown below.

b54293b08bc7ddb7110a50908049922f7cfd86f14cb49a4de4e28353a6cc6d8
d17c1fd5c6d87635daf8035fe4a0918d1eaa29708eae37cb797cae08c19ef887

The same receipt binds aggregate results from the pinned-Lean edge audit. The model-facing prompts contain the public rows, never the incidence labels.

310 An earlier protocol anticipated a fully frozen manual panel with per-call timestamps, attestations,
 311 and closure receipts. The material actually supplied for this study is one PDF of visible responses,
 312 not those richer capture forms. We do not manufacture the missing evidence or inherit claims
 313 from the unused protocol. The evaluation therefore supports deterministic score reproduction and a
 314 prior dataset commitment, but not independent verification of fresh-chat use, interface settings, call
 315 chronology, or the exact serving checkpoints.

316 D Pinned-Lean replay audit

317 The source Parquet graph is convenient for mining but is not accepted as its own verification. A
 318 second program builds one Lean input stream in a clean MATHLIB checkout and imports Mathlib.
 319 For every displayed source declaration it checks that the constant exists and has the expected
 320 declaration kind. For every target it reads `ConstantInfo.value?` with opaque values enabled,
 321 obtains `Expr.getUsedConstantsAsSet`, intersects that set with the 16 displayed candidate names,
 322 and compares the result exactly with the stored row. It also repeats the cross-statement `Expr.eqv`
 323 checks.

324 The Python wrapper binds the public and label commitments, checks the clean MATHLIB commit,
 325 verifies Lean 4.33.0 and its compiler commit, and emits only aggregate counts. This keeps private
 326 declaration names out of a public audit log. Negative tests mutate a label edge, each commitment, the
 327 source inventory, and the environment; every mutation must fail closed.

Table 4: Aggregate pinned-Lean audit of final bytes.

Check	Development	Blind
Episodes	30/30	60/60
Targets re-extracted exactly	240/240	480/480
Stored / re-extracted displayed edges	386/386	814/814
Changed targets	0	0
Added / removed edges	0/0	0/0
Empty target incidence	0	0
Expression-equivalent statement collisions	0	0

328 The audit verified clean Mathlib commit `db584cd6d46c92f209a44c0f1c829460d327499d`, Lean
 329 4.33.0 compiler commit `d8b18978322de05a8f3dba51ef03cf5461676c17`, all expected declara-
 330 tion kinds, and exact public/private commitments. A separate static audit re-hashed all 256 Parquet
 331 shards and reproduced every final development and blind output byte-for-byte.

332 E Response intake and scoring protocol

333 E.1 Direct rows

334 The score source is a 15-page PDF entitled *Models and their responses*. Its section headings supply
 335 the five visible consumer labels used in the paper. The document contains twelve intended five-
 336 episode direct responses per row, except for the two anomalies described below. It contains no
 337 account-tier, system-prompt, token-usage, latency, screenshot, or provider-log evidence. Accordingly,
 338 we make no claim about an underlying API checkpoint and do not infer operational facts that are
 339 absent from the document.

340 Transcription removes layout whitespace that the PDF extractor inserts inside identifiers, decodes
 341 complete JSON envelopes, and indexes selections by explicit `MLP4B.xxxx` keys. The expected key
 342 ranges are `D01=0000–0004` through `D12=0055–0059`. Visual order is irrelevant. Every selection
 343 must contain three distinct in-pool IDs. A missing or malformed whole response block gives all five
 344 episodes zero while retaining them in the denominator. This strict rule avoids silently improving an
 345 output by editorial repair. The canonical machine-readable transcription and source-document hash
 346 are released beside the scorer.

347 GPT contains `D01–D11`, a second `D11` response, and no `D12`. We retain the first `D11` occurrence; the
 348 only changed item in the second version has identical exact and secondary scores, so this convention
 349 does not affect any reported number. Qwen 3.8 contains the `D12` keys but omits the envelope’s

outer opening brace. We count that entire block invalid; adding the brace would increase its exact score from 12/60 to 15/60, but that repaired value is not reported. All other blocks decode and align uniquely by episode key.

E.2 Ordered-support diagnostic

Four additional GPT SOL blocks provide a ranked candidate-support list for every target in D04, D05, D06, and D12. The analysis truncates each list to its first $q = 2$ entries and enumerates all 560 portfolios against the predicted incidence, selecting the lexicographically first maximizer. This postprocessor uses no gold label. After label release, full and truncated support maps are compared with the direct-incidence graph using micro precision, recall, and F1. The same 20 episode IDs permit a descriptive comparison with direct prompting. Because the PDF does not establish fresh-chat use, counterbalanced chronology, or an absence of intervening feedback, we do not call the difference a causal prompting effect and report no significance test.

F Complete results

F.1 Complete score table

Row	Exact	Valid	Mean $F/8$	Mean F/F^*	Edge recall
SHA-256 deterministic draw	0/60	60/60	2.30/8	37.1%	18.9%
Character TF-IDF	0/60	60/60	3.77/8	60.4%	29.5%
GPT SOL 5.6 Pro	25/60	55/60	4.98/8	79.6%	38.9%
DeepSeek Instant + DeepThink	1/60	60/60	4.20/8	67.1%	34.9%
DeepSeek Expert + DeepThink	7/60	60/60	4.60/8	73.4%	36.9%
Qwen 3.8 Max-Thinking	12/60	55/60	4.32/8	68.9%	34.0%
Qwen 3.7 Plus-Thinking	6/60	60/60	4.05/8	64.6%	31.8%
GPT SOL support $q = 2$	1/20	20/20	4.60/8	74.8%	34.5%

Table 5: Complete fixed-denominator scores. Invalid responses contribute zero to every metric. The analytic uniform-random expectation is 0.14 exact episodes, 2.35/8 mean coverage, 37.6% mean F/F^* , and 18.75% edge recall.

For the five consumer rows, item-wise Wilson 95% intervals for exact accuracy are, in table order, 30.1–54.3%, 0.3–8.9%, 5.8–22.2%, 11.8–31.8%, and 4.7–20.1%. Resampling the twelve five-item response blocks gives descriptive 95% percentile intervals of 30.0–53.3%, 0.0–5.0%, 5.0–18.3%, 8.3–33.3%, and 1.7–20.0%. These intervals summarize sensitivity within this fixed challenge set; they are not estimates for model families.

F.2 Block-level exact scores and response alignment

Row	D01	D02	D03	D04	D05	D06	D07	D08	D09	D10	D11	D12
GPT SOL	2	2	2	2	1	1	4	2	3	3	3	0
DeepSeek Instant	0	0	0	0	0	0	0	0	1	0	0	0
DeepSeek Expert	1	0	0	0	1	0	1	1	0	1	2	0
Qwen 3.8 Max	1	1	1	0	0	0	1	3	3	0	2	0
Qwen 3.7 Plus	0	0	0	0	0	0	1	2	1	0	0	2

Table 6: Exact optima per five-episode direct block. Each row sums to the main-table numerator.

The source document does not always present blocks in numerical order. The alignment is recovered from the explicit episode keys. GPT supplies D01–D11, then a second D11 variant, and no D12. Both D11 variants score 3/5 and have the same secondary metrics; the first is retained as the canonical transcription. DeepSeek Instant and Expert supply D01–D12. Qwen 3.8 supplies D01,D03,D02,D04,D06,D05,D08,D07,D09,D10,D12,D11; its D12 text is malformed because the outer opening brace is absent. Qwen 3.7 supplies D02,D01,D03,D04,D08,D07,D10,D09,D05,D06,D12,D11. No positional guess is used.

377 **F.3 Structured-support diagnostic**

378 The four support blocks cover D04, D05, D06, and D12. The fixed $q = 2$ postprocessor produces
379 block exact counts 1, 0, 0, 0, compared with direct counts 2, 1, 1, 0 on the same IDs. The corre-
380 sponding exact-accuracy changes are $-20, -20, -20, 0$ percentage points. Full ranked support has
381 micro precision/recall/F1 of 72.5/75.6/74.0%; after retaining two candidates per target the figures
382 are 76.8/68.7/72.6%. Because the direct D12 response is missing, its secondary-score contrast is
383 dominated by the strict zero and should not be read as a clean prompt ablation.

384 **F.4 Provenance boundary**

385 The public test file and label digest were recorded on 1 September 2026. The score source is the
386 single author-supplied PDF received on 2 September 2026. Their SHA-256 digests, in that order, are:

```
387 b54293b08bc7d8bb7110a50908049922f7cfd86f14cb49a4de4e28353aecc6d8  
388 d17c1fd5c6d87635daf8035fe4a0918d1eaa29708eae37cb797cae08c19ef887  
389 2f20fbc4b3eb3b87c4b693058fc5744a947de04e19a2eca70f0ea5c0d54dde3f
```

390 The archive preserves this PDF, the episode-key transcription, and the scoring outputs. It does
391 not contain provider-side request logs, per-chat timestamps, or independently verified deployment
392 screenshots. We therefore describe the rows as externally captured consumer responses with a
393 pre-existing dataset commitment, not as checkpoint-reproducible or cryptographically escrowed
394 runs.

395 **G Design history: excluded earlier captures**

396 Earlier LemmaPortfolio prototypes are not pooled with this benchmark. They used smaller candidate
397 pools, a two-item budget, one privileged exact pair, and post-release inspection. Their web-interface
398 captures and local quantized-model captures differ in routing, prompting, context allocation, runtime,
399 and item set. In particular, the preserved evidence for an earlier 21/30 development capture does not
400 establish the GPT-4 identity originally associated with it. We retain the number only as design history
401 and make no GPT-4 result claim.

402 Likewise, an old local row labelled “DeepSeek” sent 15 episodes through one shared reasoning
403 budget and repeatedly fell back after parser failures. It is not comparable to a fresh consumer-web
404 deployment and is not evidence that the DeepSeek family scores two points. We also explored
405 fresh-item local Q4 captures while redesigning the protocol, but they use a different inference surface
406 and do not enter the final panel. No earlier score or response is reused.

407 **H Interpretation and limitations**

408 The benchmark has four main boundaries.

409 First, $D(t)$ is historical. A declaration can be logically useful without appearing in the stored proof,
410 and a stored dependency can be replaceable. The task therefore measures dependency compression,
411 not causal utility. UseFor, for example, studies reusability and contribution to proof likelihood in a
412 self-play setting [Kasriel et al., 2025]; that is a complementary and stronger utility question.

413 Second, this is a closed pool with uniform costs. Real systems search much larger libraries and
414 may assign different context or retrieval costs to declarations. The coverage objective is monotone
415 submodular, connecting the task to classical maximum-coverage optimization [Nemhauser et al.,
416 1978], but $K = 16$ and $B = 3$ let us enumerate exactly rather than rely on approximation guarantees.

417 Third, the stress split is conditioned to defeat fixed diagnostics. Its score is useful for comparing
418 declared methods on this challenge distribution, not for estimating the frequency of hard modules
419 in all of MATHLIB. Raw counts are primary. Item-wise Wilson intervals show binomial-scale
420 variation under an exchangeable-item working model, while whole-block bootstrap intervals show
421 sensitivity to resampling the fixed prompt groups. Neither estimates population performance or
422 model-generation variance. The 20-item intervention has only four supplied five-item blocks and
423 supports no significance claim.

424 Fourth, consumer-chat deployments hide routing, system prompts, and decoding, and the supplied
425 PDF contains no per-call metadata. The rows should not be read as a provider or architecture ranking.
426 There is no human-expert baseline, and one response per item is not an estimate of generation
427 variance. Codex agents assisted benchmark design, evaluation code, response transcription, and
428 manuscript development; the authors checked the scores and remain responsible for every claim.

429 **I Artifact map and reproduction**

430 The anonymous archive is organized so that score reproduction is distinct from the more expensive
431 reconstruction of the benchmark:

432 `data/` development and test public rows and released labels;
433 `responses/` the source PDF, canonical direct and support transcriptions, and the alignment/anomaly
434 ledger;
435 `results/` fixed baseline selections and machine-readable aggregate, item, and block scores;
436 `provenance/` the dataset commitment and aggregate construction and pinned-Lean receipts;
437 `development_evidence/` the 15-item difficulty capture and its decision receipt;
438 `construction/` the benchmark builder, tests, and pinned-Lean audit wrapper;
439 `tools/` a standard-library scorer and manifest builder;
440 `verify_release.py` the one-command release verifier; and
441 `paper/` anonymous sources and the checked anonymous PDFs.

442 The one-command verifier uses only the Python standard library. It checks the published hashes,
443 reconstructs all optima from the released incidence rows, and recomputes every model, baseline, and
444 development-check score without calling a model, Lean, or a paid service. The archive also supplies
445 the pinned-Lean audit wrapper and tests. Repeating that extraction additionally requires Lean 4.33.0
446 and the named MATHLIB snapshot; the original mining search requires the separately obtainable 256
447 Parquet shards. Exact commands and a short path for a nontechnical collaborator are in the archive
448 README.